

Sistema de Recomendação de Cursos

Integrantes:

-Lucca Borges RM554608

-Ruan Vieira RM557599

-Rodrigo Carnevale RM55814

Sumário

1. Formulação do Problema.....	3
2. Visão Geral das Funções.....	3
2.1 Função mostrar(texto).....	3
2.2 Função montar_lista_cursos(area_foco).....	4
2.3 Função coletar_perfil_usuario().....	4
2.4 Função calcular_relevancia_cursos(cursos, area_foco, prioridade, estilo).....	4
2.5 Função merge_sort_lista(lista, chave).....	5
2.6 Função montar_plano_otimo(cursos, horas_max, orcamento_max).....	5
2.7 Função executar_sistema().....	5
3. Recursão e Memoização na Solução.....	6
4. Estrutura de Saída e Relatórios.....	6

1. Formulação do Problema

O objetivo do sistema é montar um plano de estudos em Inteligência Artificial para um profissional, respeitando restrições de tempo disponível (horas de estudo) e orçamento máximo, maximizando a relevância total dos cursos escolhidos.

Entradas principais:

- Área de atuação do profissional (saúde, tecnologia, gestão, educação ou todas).
- Prioridade em relação à IA (carreira, atualização ou transição).
- Estilo de estudo (rápido, profundo ou equilibrado).
- Horas máximas de estudo disponíveis (horas_max).
- Orçamento máximo disponível para investimento em cursos (orcamento_max).

Saídas principais:

- Catálogo de cursos personalizados com impacto ajustado (impacto_ajustado).
- Catálogo ordenado por impacto ajustado, utilizando merge sort recursivo.
- Trilha de cursos selecionados pelo algoritmo da mochila 0/1 (plano ótimo).
- Resumo em formato de relatório: horas usadas, orçamento gasto e relevância total.

2. Visão Geral das Funções

A seguir, cada função do código é explicada em detalhes, incluindo seu papel na solução, parâmetros de entrada, retornos e como se relaciona com recursão, memoização e o algoritmo da mochila.

2.1 Função `mostrar(texto)`

A função `mostrar(texto)` é uma função auxiliar de interface. Ela recebe uma string e divide o texto em linhas, usando uma função recursiva interna para imprimir cada linha no console.

Para atender ao requisito de recursão e memoização, a função utiliza `@lru_cache` em uma função interna `get_linha(i)`, que memoriza o conteúdo de cada linha. Em seguida, a função recursiva `imprimir(i)` imprime a linha `i` e chama a si mesma com `i+1` até alcançar o final da lista de linhas.

Apesar de ser uma função de saída, ela foi implementada com recursão e memoização para manter consistência com o enunciado da disciplina.

2.2 Função montar_lista_cursos(area_foco)

Responsável por construir o catálogo de cursos. A função recebe a área de foco escolhida pelo usuário (por exemplo, 'saúde', 'tecnologia', 'gestão', 'educação' ou 'todas') e retorna uma lista de dicionários, onde cada dicionário representa um curso com os campos: curso, área, horas, preço e impacto base.

Os cursos estão organizados em um dicionário CURSOS_BASE, com várias opções por área. Somando as áreas, o catálogo possui mais de 20 cursos, atendendo ao requisito de trabalhar com pelo menos vinte informações.

A função utiliza uma função interna cursos_por_area(area), decorada com @lru_cache, que devolve os cursos de uma área específica. Outra função interna recursiva, juntar_areas(idx), concatena os cursos das áreas escolhidas, avançando recursivamente no índice até que todas as áreas tenham sido processadas.

Dessa forma, montar_lista_cursos utiliza recursão e memoização para construir o catálogo final.

2.3 Função coletar_perfil_usuario()

Esta função cuida da interação inicial com o usuário e coleta os parâmetros necessários para o problema. Ela trabalha com uma lista de perguntas, que incluem: área de atuação, prioridade em relação à IA, estilo de estudo, horas máximas e orçamento máximo.

Internamente, a função define uma função recursiva perguntar(i, respostas), que processa uma pergunta por vez. No caso base, quando i chega ao final da lista de perguntas, a função retorna o dicionário de respostas. No caso recursivo, ela apresenta a pergunta, lê a resposta do usuário, trata os campos numéricos e chama perguntar(i+1, respostas).

Para memoização, é utilizado um dicionário memo_respostas, que guarda cópias parciais das respostas já obtidas. Isso demonstra conceitos de recursão e memoização aplicados ao fluxo de entrada de dados.

2.4 Função calcular_relevancia_cursos(cursos, area_foco, prioridade, estilo)

A função calcular_relevancia_cursos recebe a lista de cursos e o perfil do usuário (área de foco, prioridade e estilo de estudo) e calcula um novo atributo para cada curso: impacto_ajustado.

O impacto_ajustado representa a relevância personalizada do curso para aquele usuário. Ele é calculado a partir do impacto base do curso multiplicado por pesos de prioridade, estilo de estudo e um bônus de área quando a área do curso coincide com a área de atuação do profissional.

Dois auxiliares com memoização são utilizados: peso_prioridade(p) e peso_estilo(e), ambos decorados com @lru_cache, que devolvem fatores numéricos conforme a prioridade (carreira, atualização ou transição) e o estilo de estudo (rápido, profundo, equilibrado).

A função recursiva interna `processar_indice(i)` percorre a lista de cursos, calculando `impacto_ajustado` para cada um e montando uma nova lista resultante. Por fim, o `impacto_ajustado` é limitado a no máximo 10, simulando uma nota máxima. O resultado é uma lista de cursos com esse novo campo, que será utilizado na ordenação e na mochila.

2.5 Função `merge_sort_lista(lista, chave)`

A função `merge_sort_lista` implementa o algoritmo de Merge Sort de forma recursiva, com memoização, para ordenar uma lista de dicionários conforme uma chave numérica (por exemplo, `impacto_ajustado`).

Ela define uma função interna `sort_interval(start, end)`, decorada com `@lru_cache`, que ordena o subintervalo da lista entre `start` (inclusivo) e `end` (exclusivo).

No caso base, quando o tamanho do intervalo é 0 ou 1, a sublista já está ordenada. No caso recursivo, a função divide o intervalo ao meio, ordena recursivamente as metades esquerda e direita e, em seguida, faz o merge das duas listas ordenadas, comparando o valor da chave nos elementos.

O resultado é uma lista ordenada em ordem decrescente pela chave escolhida, atendendo ao requisito de usar quick ou merge sort para ordenar dados em um dataframe.

2.6 Função `montar_plano_otimo(cursos, horas_max, orcamento_max)`

A função `montar_plano_otimo` é responsável por aplicar a ideia da mochila 0/1 (knapsack) ao problema. Ela recebe a lista de cursos, as horas máximas e o orçamento máximo e retorna o melhor conjunto de cursos que maximiza a soma dos impactos ajustados respeitando as restrições.

Internamente, a função define `knapsack(i, horas_restantes, orcamento_restante)`, decorada com `@lru_cache(maxsize=None)`. Cada estado da programação dinâmica é representado por um índice `i` de curso e pelas quantidades de `horas_restantes` e `orcamento_restante`.

No caso base, se `i` alcança o final da lista ou se as restrições de horas ou orçamento chegam a zero, não é possível adicionar mais cursos e o valor retornado é 0. No caso recursivo, a função calcula o melhor valor sem pegar o curso atual e, se ele couber nas restrições, o valor alternativo pegando o curso. O maior dos dois é retornado, caracterizando a escolha ótima da mochila 0/1.

Após calcular o valor máximo, uma função recursiva de reconstrução percorre novamente os estados e descobre quais cursos foram efetivamente escolhidos na solução ótima. Por fim, são calculados as horas usadas, o gasto total e o impacto máximo.

2.7 Função `executar_sistema()`

A função `executar_sistema` é o ponto de entrada da aplicação. Ela coordena todas as demais funções por meio de um menu interativo, implementado com um loop recursivo interno.

Um dicionário cache é utilizado para armazenar o perfil do usuário, o catálogo gerado, o catálogo ordenado e o plano ótimo. Isso evita recomputar informações desnecessárias e funciona como uma forma de memoização no nível da aplicação.

O menu oferece as opções: (1) informar/atualizar o perfil, (2) gerar o catálogo personalizado e ordenar, (3) calcular o plano ótimo pela mochila e (0) sair. Cada opção chama as funções correspondentes, exibindo os resultados em DataFrames e em texto.

3. Recursão e Memoização na Solução

A solução foi construída de forma a demonstrar recursão e memoização em todas as partes centrais do código:

- mostrar: recursão nas linhas, memoização nas linhas com `get_linha(i)`.
- montar_lista_cursos: recursão ao juntar áreas, memoização por área em `cursos_por_area`.
- coletar_perfil_usuario: recursão na sequência de perguntas, memo das respostas parciais.
- calcular_relevancia_cursos: recursão para percorrer cursos; memoização para pesos de prioridade e estilo.
- merge_sort_lista: recursão do Merge Sort; memoização dos intervalos ordenados.
- montar_plano_otimo: recursão clássica da mochila 0/1, memoização dos estados (`i`, horas, orçamento).
- executar_sistema: loop recursivo para o menu e cache para reaproveitar resultados.

Com isso, a implementação atende diretamente ao requisito do enunciado de utilizar recursão e memoização em todas as funções criadas.

4. Estrutura de Saída e Relatórios

A saída do sistema foi estruturada para facilitar a interpretação do professor e do usuário final. Após a execução das etapas, são apresentados:

- Um DataFrame com o catálogo personalizado de cursos, incluindo o impacto ajustado para o perfil do usuário.
- Um DataFrame apenas com os cursos selecionados pela mochila, representando o plano ótimo de estudos.
- Um resumo textual com as horas totais utilizadas, o orçamento gasto e a relevância total obtida.

Essa combinação de tabelas e resumos textuais atende ao requisito de apresentação de resultados e relatórios, tornando clara a relação entre as entradas do usuário, o processamento pelo algoritmo e as decisões finais do sistema.